

GitLens Pro

GitHub Repository Analysis Platform

Software Requirements & Design Document

Version 1.2 Final | March 2026

Author: Ben Meddeb

CONFIDENTIAL

Built with Go | Uber FX | MySQL | GitHub OAuth

v1.2 Final: Review/Audit, notifications, path ownership, tri-state access revalidation, review state machine, race-proof job dedup, suspended-repo APIs, and contract cleanup.

Table of Contents

PART I: PRODUCT REQUIREMENTS	3
1. Executive Summary	3
2. Goals & Non-Goals	3
3. Functional Requirements	4
4. Non-Functional Requirements	9
PART II: ARCHITECTURE DECISIONS	12
5. Technology Stack	12
6. System Architecture	13
PART III: DATA MODEL	17
7. Data Models	17
PART IV: API & INTEGRATION	28
8. API Design	28
9. Configuration	31
PART V: CORE ALGORITHMS & SECURITY	33
10. Authentication Flow	33
11. Fork Analysis Algorithm	33
12. Security	35
PART VI: OPERATIONS & DELIVERY	36
13. Testing Strategy	36
14. Deployment	36
15. Development Milestones	37
16. Open Decisions	37
Appendix A: Glossary	38
Appendix B: v1.1 -> v1.2 Changelog	38

PART I: PRODUCT REQUIREMENTS

1. Executive Summary

GitLens Pro is a multi-user web application for deep analysis and review of GitHub repositories. Users clone repositories locally, inspect commit histories, analyze contributors, run blame, view diffs, analyze fork evolution, and conduct asynchronous code reviews and audits. All classification and review routing is rule-based in v1.2.

Built with Go and Uber FX for dependency injection, backed by MySQL, integrated with the GitHub API and OAuth.

2. Goals & Non-Goals

2.1 Goals

- Provide a web-based platform for deep repository analysis beyond what GitHub offers natively
- Enable local cloning and git-level inspection (commits, blame, diffs, contributors)
- Build a fork analysis engine mapping repository evolution across fork networks
- Provide asynchronous code review and audit workflows anchored to commits, ranges, and fork deltas
- Deliver a multi-user system with GitHub OAuth authentication and shared workspaces
- Use Uber FX for clean dependency injection, lifecycle management, and modular architecture

2.2 Non-Goals (v1.2)

- Real-time co-editing, live cursors, or synchronous collaborative editing of repositories
- CI/CD pipeline integration, build status monitoring, or automatic merge gating. Manual review decisions and verification requests are in scope
- Repository hosting, branch protection enforcement, pull request orchestration, or merge/landing workflows
- Support for non-GitHub providers (GitLab, Bitbucket) in the initial release
- AI/ML-powered analysis or review summarization. Classification and review routing remain rule-based in v1.2

- Public-facing API. The REST API remains an internal contract consumed by the first-party frontend
- Reply-by-email, inbound email parsing, or patch submission by email

3. Functional Requirements

3.1 Authentication & User Management

FR-AUTH-01: Users authenticate exclusively via GitHub OAuth 2.0 (Authorization Code flow). No local username/password.

FR-AUTH-02: OAuth scope: read:user, user:email, repo. user:email required for GET /user/emails. If declined, email stored as NULL.

FR-AUTH-03: On first login, User record created from GitHub profile. On subsequent logins, display_name, avatar_url, email updated.

FR-AUTH-04: GitHub OAuth access token stored AES-256-GCM encrypted in MySQL (VARBINARY).

FR-AUTH-05: Sessions via signed HTTP-only SameSite=Lax Secure cookies. Cookie contains session ID only. State in MySQL sessions table.

FR-AUTH-06: All routes except /auth/*, /healthz, /readyz require authenticated session. Unauthenticated = 401.

FR-AUTH-07: POST /auth/logout destroys session and clears cookie.

3.2 Repository Management

FR-REPO-01: User adds a repository by providing a GitHub URL or owner/name pair.

FR-REPO-02: System validates repo via GitHub API using user's token. Private repos: token lacks access → "not found or not accessible."

FR-REPO-03: Adding creates a UserRepository association. If a canonical Repository already exists and no async work is required, the attach completes synchronously and returns HTTP 201 with Repository + UserRepository data. If the canonical Repository does not yet exist, or local clone work is required, the request enqueues the required async job and returns HTTP 202.

FR-REPO-04: Users see only repos in their workspace. Removing deletes UserRepository. Canonical record + clone deleted when zero refs remain (nightly).

FR-REPO-05: Users can trigger sync (git pull) as async job.

FR-REPO-06: All async operations return HTTP 202 with a Job resource and Location header. Dedup hits use the same contract and include X-Job-Deduplicated: true.

FR-REPO-07: Access to a private repository is revalidated on add, sync, repo-open, and review-open. Confirmed denial suspends the association. Indeterminate checks do not suspend and preserve the last-known access state.

FR-REPO-08: Review and audit objects are repo-scoped and visible only to users with current repository access.

3.3 Private Repository Access Revocation

FR-ACCESS-01: Private-repository revalidation yields one of three results: accessible, denied, or indeterminate.

FR-ACCESS-02: On login, the system re-checks all private repositories in the workspace with a lightweight GitHub repository metadata request. Accessible sets the association active. Confirmed denial sets the association suspended. Indeterminate results do not change state.

FR-ACCESS-03: On repo-open or review-open, re-check when the last successful verification is older than 1 hour. Denied blocks content and suspends access. Indeterminate preserves the last-known state and surfaces a warning banner.

FR-ACCESS-04: Suspended repositories are excluded from the default listing but remain visible via GET /api/repos?state=suspended|all. Users may trigger POST /api/repos/:id/recheck-access after re-authentication or remove the association.

FR-ACCESS-05: Check granularity: login (full sweep), repo/review-open (if >1h stale), sync (always), email delivery (always).

FR-ACCESS-06: Public repositories are exempt from re-validation.

3.4 Commit History & Browsing

FR-COMMIT-01: Paginated commit log with author, date, message, change stats.

FR-COMMIT-02: Filtering by: author email/name, date range, file path, message keyword.

FR-COMMIT-03: Each commit links to detail view with full diff.

FR-COMMIT-04: Diff: unified and side-by-side, syntax highlighting, expandable context.

FR-COMMIT-05: Merge commits diff against first parent by default.

FR-COMMIT-06: Commit diff responses expose stable hunk and line anchors for inline review comments.

FR-COMMIT-07: Commit detail views include linked review and audit summaries when the commit is present in any review revision.

3.5 Contributor Analysis

FR-CONTRIB-01: Aggregated stats per contributor: commits, insertions, deletions, active range.

FR-CONTRIB-02: Contributors identified by canonical identity from (name, email) pairs. GitHub noreply resolved.

FR-CONTRIB-03: File ownership: per file, % of lines per contributor via blame.

3.6 Blame Analysis

FR-BLAME-01: git-blame on any file at any ref (default: HEAD of default branch).

FR-BLAME-02: Line-by-line attribution: author, SHA, date, first line of message.

FR-BLAME-03: Cache keyed by (repo_id, file_path, commit_sha). Entries valid while SHA reachable. Nightly cleanup for stale.

FR-BLAME-04: Files >10,000 lines truncated with warning.

3.7 Fork Network Analysis

Primary differentiating feature. Algorithm details in Section 11.

FR-FORK-01: Fetch and display all forks via GitHub API (paginated, up to configured max).

FR-FORK-02: Users select forks for deep analysis. Deep analysis clones fork and compares against upstream default branch.

FR-FORK-03: Comparisons always against upstream default branch. Arbitrary branch comparison deferred.

FR-FORK-04: Per fork: ahead/behind counts, merge-base SHA, divergence date, unique commits, classified changes.

FR-FORK-05: Cherry-picks and rebases detected and flagged (Sections 11.3, 11.4).

FR-FORK-06: All classification rule-based. No ML/LLM in v1.2.

FR-FORK-07: Fork evolution report: changes across forks, duplicate work highlights.

FR-FORK-12: Fork reports expose machine-readable review-suggestion objects for unique fork changes and similarity clusters.

FR-FORK-13: Users can create an audit from a fork change or similarity cluster with the relevant commit set pre-populated as the first review revision.

3.8 Code Review & Audit

FR-REVIEW-01: Users can create a Review for any repo in their workspace from: single commit, explicit commit range, base/head ref pair, analyzed fork delta relative to upstream, or fork-similarity suggestion.

FR-REVIEW-02: A review has kind = review | audit and status = draft | open | changes_requested | accepted | resolved | closed.

FR-REVIEW-03: A review stores title, summary, author, repository, origin metadata, and a pointer to the latest revision.

FR-REVIEW-04: A review revision stores the exact diff basis: base commit SHA, head commit SHA, optional base/head refs.

FR-REVIEW-05: Review diff rendering reuses the existing diff engine. Supports unified/side-by-side, syntax highlighting, expandable context, file navigation.

FR-REVIEW-06: Diff responses for review views expose stable file/hunk/line anchors suitable for inline comments.

FR-REVIEW-07: Inline comments anchored by file_path, side, start_line, end_line, revision_id. Overall comments (no file anchor) also supported.

FR-REVIEW-08: Inline comments are drafts by default, visible only to their author until published.

FR-REVIEW-09: Publishing drafts submits current user's drafts as a single batch with optional summary comment and optional decision (comment, changes_requested, accepted, resolved, or closed).

FR-REVIEW-10: Published comments support threading, mentions, resolve, and reopen.

FR-REVIEW-11: Participant roles: author, reviewer, blocking_reviewer, subscriber.

FR-REVIEW-12: Participant state: pending, commented, changes_requested, accepted, watching.

FR-REVIEW-13: A review cannot enter accepted while any blocking_reviewer is pending or changes_requested.

FR-REVIEW-14: New revisions re-anchor prior inline comments onto new diff. Unmappable comments marked orphaned but retained.

FR-REVIEW-15: An audit may be created from a fork-only commit, classified fork change, or similar-changes-across-forks cluster.

FR-REVIEW-16: Review/audit objects do not create GitHub PRs, do not enforce branch protection, do not merge code.

3.8.1 Review Authorization Matrix

Create review/audit: Any user with active repository access in the workspace. Creating an audit from a fork suggestion uses the same rule; the creator becomes the review author.

Add revision: Review author only. Other participants cannot change the diff basis.

Add/remove participants: Review author may add or remove reviewers, blocking reviewers, and subscribers. Any user may add or remove themselves as subscriber. The author cannot be removed.

Draft and publish comments: Any current participant with active repository access may create drafts. Only the draft author may publish that draft batch.

Resolve or reopen comments: Comment author, review author, or any blocking reviewer.

Submit decisions: Only reviewers and blocking reviewers may submit accepted or changes_requested. Authors cannot accept their own review. Review author or any blocking reviewer may close, resolve, or reopen a review.

Subscriber role: Subscribers may comment and follow updates but have no decision rights.

3.8.2 Review Status & Transition Rules

State ownership: draft, resolved, and closed are explicit states. open, changes_requested, and accepted are derived from participant decisions on the latest revision.

Creation: A new review starts open by default. draft is allowed only before first reviewer notification.

New revision: Adding a revision resets reviewer and blocking_reviewer states of accepted or changes_requested back to pending for the latest revision. Subscribers remain watching; author state is unchanged. Review status returns to open.

Changes requested: A changes_requested decision from any reviewer or blocking reviewer moves the review to changes_requested.

Accepted: accepted requires at least one reviewer or blocking reviewer in accepted, no reviewer or blocking reviewer in changes_requested, and all blocking reviewers in accepted.

Resolved vs closed: resolved ends an audit or review concern without merge semantics. closed is an explicit administrative terminal state. reopen returns resolved or closed to open.

Blocking semantics: Non-blocking reviewers may place a review into changes_requested. Blocking reviewers additionally gate the transition to accepted.

3.9 Notifications & Ownership Automation

FR-NOTIFY-01: In-app notifications for: review created, revision added, reviewer added, subscriber added, draft batch published, mention, changes requested, accepted, resolved, closed.

FR-NOTIFY-02: Email notifications for same event set when recipient's preference enables email delivery.

FR-NOTIFY-03: Outbound email uses stable per-review threading headers.

FR-NOTIFY-04: Notification preferences per event type: instant, digest, web_only.

FR-NOTIFY-05: Users can suppress notifications for their own actions.

FR-NOTIFY-06: Users may follow/unfollow a review by adding/removing themselves as subscriber.

FR-NOTIFY-07: Path-owner rules auto-add users as reviewer, blocking_reviewer, or subscriber based on changed file paths and optional file extensions, but only when the target user already has active UserRepository access for that repository.

FR-NOTIFY-08: Private repo email notifications include metadata and deep link by default; raw diff hunks omitted.

FR-NOTIFY-09: Access revalidated immediately before email send. If recipient lost access, email suppressed.

FR-NOTIFY-10: Reply-by-email and inbound mail parsing deferred beyond v1.2.

FR-NOTIFY-11: Creating a path-owner rule for a user without active repository access fails validation. At evaluation time, rules targeting inactive or suspended users are skipped.

4. Non-Functional Requirements

4.1 Performance Targets

Operation	Target	Notes
Commit list (paginated)	< 200ms p95	After indexing into MySQL
Single commit + diff	< 500ms p95	From local clone via go-git
Blame (cache hit)	< 100ms p95	From MySQL cache
Blame (cache miss, <5K lines)	< 3s p95	Computed then cached
Fork list (GitHub API)	< 2s p95	Depends on GitHub latency
Fork deep analysis	< 5 min per fork	Async job
Review detail + diff (≤ 200 files)	< 700ms p95	Reuses diff engine
Publish draft batch (≤ 100 comments)	< 300ms p95	Excluding email dispatch
Notification enqueue	< 100ms p95	In-app + email queue
Instant email delivery	median < 60s, p95 < 5 min	From enqueue to SMTP send
Page load (HTML)	< 1s p95	Server-rendered

4.2 Capacity Limits

Resource	Limit	Behavior When Exceeded
Max repo size (clone)	2 GB on disk	Reject with error

Max files in single diff	500 files	Truncate with warning
Max blame file size	10,000 lines	Truncate with warning
Max forks for listing	1,000	Paginate; warn if exceeded
Max forks for deep analysis	50 per repo	User must select
Concurrent clone jobs	5	Queue FIFO
Concurrent analysis jobs	10	Queue FIFO
Max repos per user	100	Quota error
blame_data per row (uncompressed)	5 MB	Refuse file
ForkChange per fork	10,000	Summarize older
Max review revisions per review	50	Reject new revision
Max participants per review	100	Reject add
Max blocking reviewers per review	10	Reject add
Max draft comments per publish batch	200	Reject publish
Max comment body length	10,000 chars	Reject with 422 validation error
Notification retention	90 days	Purge nightly

4.3 Storage & Retention

- **Clone storage:** storage/repos/{repo_id}/. Orphaned = nightly delete.
- **Fork storage:** storage/forks/{fork_id}/.
- **Blame cache:** Nightly cleanup for unreachable SHAs and entries >30 days.
- **Job records:** 90-day retention.
- **Reviews, revisions, participants, comments:** Retained for lifetime of canonical Repository record. Cascade-deleted when repo is cleaned up.
- **Notifications:** 90-day retention, then purged.
- **Digest artifacts:** 30-day retention.
- **Disk monitoring:** /readyz reports disk space. < 1 GB free → reject new clones.

4.4 Resilience & Recovery

- **Restart recovery:** Running jobs reset to pending. Denormalized status fields reset accordingly.
- **Clone interruption:** Partial clones deleted. Retry from scratch.
- **GitHub rate limiting:** Per-user-token tracking via GitHubClientFactory. Remaining < 50 -> pause non-critical work. 429 or rate-limit 403 -> sleep until reset. Access revalidation classifies rate-limit and transient failures as indeterminate, never denied.
- **Database:** GORM pool with reconnect. Only transient, idempotent connection failures on safe read/query paths or job dequeue/update steps are retried once after 1s. Transactional writes are not blindly retried.
- **Email delivery:** Failed sends retried 3 times with exponential backoff. After exhaustion, email_state = failed.

PART II: ARCHITECTURE DECISIONS

Decisions tagged **[LOCKED]** or **[DEFERRED]**.

5. Technology Stack

Component	Decision	Status
Language	Go 1.22+	LOCKED
DI Framework	Uber FX	LOCKED
HTTP Router	Chi v5	LOCKED
Database	MySQL 8.0+	LOCKED
ORM	GORM v2	LOCKED
Auth	GitHub OAuth 2.0 (Authorization Code)	LOCKED
OAuth Scopes	read:user, user:email, repo	LOCKED
Sessions	MySQL sessions table, signed cookies	LOCKED
Git Ops	go-git v5	LOCKED
GitHub API	go-github v60+, x/oauth2	LOCKED
Config	Viper	LOCKED
Logging	Zap	LOCKED
Migrations	golang-migrate v4	LOCKED
Frontend Architecture	Server-rendered HTMX	LOCKED
Template Engine	html/template vs Templ	DEFERRED
Email Delivery	SMTP / SES / Postmark / Resend	DEFERRED

Session storage: Locked to MySQL. Redis rejected for v1.2 to minimize infrastructure dependencies.

Frontend architecture: Locked to server-rendered HTMX. Only the template engine choice (html/template vs Templ) remains deferred for Phase 1.

6. System Architecture

6.1 Uber FX Module Structure

Authoritative module definition. Dependency graph in 6.3 is derived.

FX Module	Provides (fx.Provide)	Depends On
config	AppConfig	(none)
logger	*zap.Logger	AppConfig
database	*gorm.DB, migration runner	AppConfig, *zap.Logger
auth	OAuthHandler, SessionMiddleware, SessionStore	AppConfig, *gorm.DB, UserService, *zap.Logger
user	UserService, UserRepository	*gorm.DB, *zap.Logger
repository	RepoService, UserRepoService, RepoRepository	*gorm.DB, *zap.Logger, GitService, GitHubClientFactory
gitclient	GitService	AppConfig, *zap.Logger
githubclient	GitHubClientFactory	AppConfig, *zap.Logger
analysis	CommitAnalyzer, BlameAnalyzer, ForkAnalyzer	GitService, *zap.Logger
review	ReviewService, ReviewRepository, ReviewDiffService	*gorm.DB, GitService, RepoService, *zap.Logger
notification	NotificationService, NotificationRepository, PathOwnerService	*gorm.DB, ReviewService, RepoService, *zap.Logger
mailer	Mailer, TemplateRenderer, NotificationWorker	AppConfig, NotificationService, *zap.Logger
jobs	JobService, JobRepository, CloneWorker, AnalysisWorker	*gorm.DB, *zap.Logger, GitService, GitHubClientFactory, RepoService, analysis.*
http	(none — fx.Invoke only)	Chi router, all Service types, SessionMiddleware

Key design decisions:

- **auth depends on UserService** (provided by user module). User create/update is called by auth during OAuth flow.

- **githubclient provides GitHubClientFactory:** Creates per-user clients from decrypted tokens. Rate limits tracked per-token, not globally.
- **http module uses fx.Invoke only:** Centralized route registration. Feature modules provide services and handlers. The http module is the only module that invokes route registration. Handler registration centralized in internal/http/.
- **jobs limited to user-visible operational jobs:** clone, sync, fork_analysis. Email dispatch uses the notification worker (mailer module), not /api/jobs.
- **review, notification, mailer** are new modules introduced in v1.2 for code review and email delivery.

6.2 FX Lifecycle Hooks

OnStart:

1. config: Load and validate YAML + env
2. logger: Init zap logger
3. database: Open MySQL pool, run migrations
4. githubclient: Validate app/client configuration, construct factory
5. jobs: Reset stale running jobs, update denormalized status, start workers
6. mailer: Start notification delivery worker and digest scheduler
7. http: Start Chi server

User token scope and repository access validation are per-request and per-job, not a global startup check.

OnStop:

1. http: Graceful shutdown (30s drain)
2. mailer: Stop notification worker, flush pending deliveries (30s)
3. jobs: Signal workers, wait (60s), mark interrupted as pending
4. database: Close pool
5. logger: Flush

6.3 Dependency Graph (Derived)

Derived from 6.1. Module table is authoritative on conflict.

```
config
└─ logger
```

```

└ database
  └ user
  └ auth (+ user)
  └ repository (+ gitclient, githubclient)
  └ review (+ gitclient, repository)
  └ notification (+ review, repository)
  └ jobs (+ all services, analysis)
└ gitclient (+ config)
└ githubclient (+ config)
└ analysis (+ gitclient)
└ mailer (+ config, notification)

```

http (fx.Invoke – centralized routes)

6.4 Application Bootstrap

```

func main() {
    fx.New(
        config.Module,
        logger.Module,
        database.Module,
        user.Module,
        auth.Module,
        repository.Module,
        gitclient.Module,
        githubclient.Module,
        analysis.Module,
        review.Module,
        notification.Module,
        mailer.Module,
        jobs.Module,
        http.Module,
    ).Run()
}

```

6.5 Directory Layout

Path	Purpose
cmd/server/main.go	FX entry point
internal/config/	Viper config, AppConfig, validation

internal/database/	MySQL pool (GORM), migration runner
internal/user/	User model, UserService, UserRepository
internal/auth/	OAuth flow, session store, session middleware
internal/repository/	Repository + UserRepository models, RepoService, UserRepoService
internal/gitclient/	go-git wrapper: clone, log, blame, diff, merge-base
internal/githubclient/	GitHubClientFactory, per-user client, rate-limit tracking
internal/analysis/	CommitAnalyzer, BlameAnalyzer, ForkAnalyzer, classification rules
internal/review/	Review, Revision, Participant, Comment models, services, diff anchoring
internal/notification/	Notification, Preference, PathOwnerRule models, services, rule eval
internal/mailer/	Email rendering, delivery worker, digest scheduler
internal/jobs/	Job model, JobService, workers
internal/http/	Chi router, centralized route registration, middleware
internal/http/handlers/	auth, repo, analysis, review, notification, jobs, admin handlers
migrations/	Versioned SQL files
config/	config.yaml, config.example.yaml
storage/repos/	Repo clones (Docker volume)
storage/forks/	Fork clones (Docker volume)

PART III: DATA MODEL

7. Data Models

7.1 User

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	Internal ID
github_id	BIGINT UNSIGNED	UNIQUE NOT NULL	GitHub user ID
username	VARCHAR(255)	UNIQUE NOT NULL	GitHub login
display_name	VARCHAR(255)		Display name
email	VARCHAR(255)	NULLABLE	Primary email (NULL if scope declined)
avatar_url	VARCHAR(512)		Avatar URL
access_token	VARBINARY(512)	NOT NULL	AES-256-GCM encrypted token
token_expiry	DATETIME	NULLABLE	Token expiry
role	ENUM('user','admin')	DEFAULT 'user'	App role
created_at	DATETIME	NOT NULL	Created
updated_at	DATETIME	NOT NULL	Updated
last_login_at	DATETIME		Last login

7.2 Session

Field	Type	Constraints	Description
id	CHAR(64)	PK	Random session ID (hex)
user_id	BIGINT UNSIGNED	FK → User NOT NULL	Owner
created_at	DATETIME	NOT NULL	Created
expires_at	DATETIME	NOT NULL	Expiry
last_active_at	DATETIME	NOT NULL	Last activity

7.3 Repository

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
github_id	BIGINT UNSIGNED	UNIQUE NOT NULL	GitHub repo ID
owner	VARCHAR(255)	NOT NULL	Owner
name	VARCHAR(255)	NOT NULL	Name
full_name	VARCHAR(512)	UNIQUE NOT NULL	owner/name
description	TEXT		Description
default_branch	VARCHAR(255)	DEFAULT 'main'	Default branch
clone_url	VARCHAR(512)	NOT NULL	Clone URL
is_fork	BOOLEAN	DEFAULT FALSE	Is fork
is_private	BOOLEAN	DEFAULT FALSE	Is private
parent_github_id	BIGINT UNSIGNED		Parent GitHub ID
local_path	VARCHAR(512)		Clone path
clone_status	ENUM('pending','cloning','ready','error')	DEFAULT 'pending'	Denormalized from Job
last_synced_at	DATETIME		Last pull
created_at	DATETIME	NOT NULL	Created
updated_at	DATETIME	NOT NULL	Updated

7.3.1 UserRepository

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
user_id	BIGINT UNSIGNED	FK → User NOT NULL	User
repo_id	BIGINT UNSIGNED	FK → Repository NOT NULL	Repo
added_at	DATETIME	NOT NULL	When added
access_state	ENUM('active','suspended')	DEFAULT 'active'	Access state
last_access_verified_at	DATETIME	NOT NULL	Last GitHub check

nickname	VARCHAR(255)		User display name
----------	--------------	--	-------------------

UNIQUE: (user_id, repo_id). Suspended repositories remain in the user's record set but are hidden from default listings; access is restored only by a successful recheck.

7.4 Job

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
type	ENUM('clone','sync','fork_analysis')	NOT NULL	Type
status	ENUM('pending','running','completed','failed','cancelled')	DEFAULT 'pending'	Status
repo_id	BIGINT UNSIGNED	FK → Repository NOT NULL	Target repo
fork_id	BIGINT UNSIGNED	FK → Fork NULLABLE	Target fork
dedup_key	VARCHAR(255)	UNIQUE NULLABLE	Present only for pending/running jobs; hash(type repo_id fork_id). NULL in terminal states.
requested_by	BIGINT UNSIGNED	FK → User NOT NULL	Requester
progress_pct	TINYINT UNSIGNED	DEFAULT 0	0–100
progress_message	VARCHAR(512)		Status text
result_summary	JSON		Completion result
error_message	TEXT		Failure detail
retry_count	TINYINT UNSIGNED	DEFAULT 0	Retries so far
max_retries	TINYINT UNSIGNED	DEFAULT 3	Max retries
created_at	DATETIME	NOT NULL	Created
started_at	DATETIME		Exec start
completed_at	DATETIME		Exec end

7.4.1 Job Dedup & Shared Access

Dedup: Pending and running jobs carry a dedup_key derived from (type, repo_id, fork_id); terminal jobs clear it to NULL. A UNIQUE index on dedup_key makes enqueue race-proof. Insert runs in a transaction; on unique-key collision the service fetches the existing active job and returns HTTP 202 with X-Job-Deduplicated: true.

Action	Who	Behavior
View job (GET /api/jobs/:id)	Any user with repo in workspace	200 OK
List own jobs (GET /api/jobs)	Current user	requested_by = current user
List repo jobs (GET /api/repos/:id/jobs)	Any workspace user	All jobs for repo
Cancel	Requester only	Others receive 403
Dedup hit	Second requester	202 + existing job + X-Job-Deduplicated: true

7.4.2 Status Denormalization

Repository.clone_status and Fork.analysis_status are denormalized from Job.status for listing queries.

- **Update rules:** Updated by worker at each job transition. On restart, if latest job reset to pending, denormalized field also reset.
- **Source of truth:** Job.status. Discrepancies corrected from latest job.

7.5 Commit

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
repo_id	BIGINT UNSIGNED	FK → Repository NOT NULL	Repo
sha	CHAR(40)	NOT NULL	SHA
author_name	VARCHAR(255)		Author
author_email	VARCHAR(255)		Email
committer_name	VARCHAR(255)		Committer

committed_at	DATETIME	NOT NULL	Date
message	TEXT		Message
files_changed	INT UNSIGNED		File count
insertions	INT UNSIGNED		Adds
deletions	INT UNSIGNED		Deletes
parent_shas	JSON		Parents

UNIQUE: (repo_id, sha). **INDEX:** (repo_id, committed_at DESC), (repo_id, author_email).

7.6 Contributor Identity / Alias / Stat

Unchanged from prior version. ContributorIdentity (canonical_email UNIQUE), ContributorAlias (observed_name, observed_email UNIQUE), ContributorStat (repo_id, identity_id UNIQUE).

7.7 Fork

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
source_repo_id	BIGINT UNSIGNED	FK → Repository NOT NULL	Upstream
github_fork_id	BIGINT UNSIGNED	UNIQUE NOT NULL	GitHub fork ID
fork_owner	VARCHAR(255)	NOT NULL	Owner
fork_name	VARCHAR(255)	NOT NULL	Name
fork_full_name	VARCHAR(512)	NOT NULL	owner/name
clone_url	VARCHAR(512)		Clone URL
local_path	VARCHAR(512)		Local path
ahead_by	INT UNSIGNED	DEFAULT 0	Ahead
behind_by	INT UNSIGNED	DEFAULT 0	Behind
merge_base_sha	CHAR(40)		Common ancestor
diverged_at	DATETIME		Divergence date
analysis_status	ENUM('listed','pending','analyzing','complete')	DEFAULT 'listed'	Denormalized from Job

	e','error')		
last_analyzed_at	DATETIME		Last analysis

7.8 ForkChange

Unchanged. UNIQUE (fork_id, commit_sha). Classification in Section 11.

7.9 BlameCache

Unchanged. UNIQUE (repo_id, file_path, commit_sha). Gzip MEDIUMBLOB. 10K cap. Smart invalidation in Section 11.6.

7.10 Review

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
repo_id	BIGINT UNSIGNED	FK → Repository NOT NULL	Repository
kind	ENUM('review','audit')	NOT NULL	Type
status	ENUM('draft','open','changes_requested','accepted','resolved','closed')	DEFAULT 'draft'	Status
author_id	BIGINT UNSIGNED	FK → User NOT NULL	Author
title	VARCHAR(512)	NOT NULL	Title
summary	TEXT		Summary
origin_type	ENUM('manual','single_commit','commit_range','fork_change','similarity_cluster')	NOT NULL	Origin
origin_payload	JSON		Origin metadata
latest_revision_id	BIGINT UNSIGNED	FK → ReviewRevision	Latest revision
created_at	DATETIME	NOT NULL	Created

updated_at	DATETIME	NOT NULL	Updated
closed_at	DATETIME	NULLABLE	Closed

INDEX: (repo_id, status, updated_at), (author_id, created_at). draft, resolved, and closed are explicit states; open, changes_requested, and accepted are derived from participant decisions on the latest revision (Section 3.8.2).

7.11 ReviewRevision

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
review_id	BIGINT UNSIGNED	FK → Review NOT NULL	Parent review
revision_number	INT UNSIGNED	NOT NULL	Sequential number
base_ref	VARCHAR(255)		Base ref name
head_ref	VARCHAR(255)		Head ref name
base_commit_sha	CHAR(40)	NOT NULL	Base SHA
head_commit_sha	CHAR(40)	NOT NULL	Head SHA
file_count	INT UNSIGNED		Files changed
insertions	INT UNSIGNED		Adds
deletions	INT UNSIGNED		Deletes
created_by	BIGINT UNSIGNED	FK → User NOT NULL	Creator
created_at	DATETIME	NOT NULL	Created

UNIQUE: (review_id, revision_number). Immutable once created.

7.12 ReviewParticipant

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
review_id	BIGINT UNSIGNED	FK → Review NOT NULL	Review
user_id	BIGINT UNSIGNED	FK → User NOT NULL	User

role	ENUM('author','reviewer','blocking_reviewer','subscriber')	NOT NULL	Role
state	ENUM('pending','commented','changes_requested','accepted','watching')	DEFAULT 'pending'	State
added_by	BIGINT UNSIGNED	FK → User NOT NULL	Added by
added_reason	ENUM('manual','path_owner_rule','mention','fork_suggestion')	NOT NULL	Reason
created_at	DATETIME	NOT NULL	Created
updated_at	DATETIME	NOT NULL	Updated

UNIQUE: (review_id, user_id). blocking_reviewer outranks reviewer; author always explicit. Participant state applies to the latest revision only; adding a new revision resets accepted and changes_requested reviewer states to pending.

7.13 ReviewComment

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
review_id	BIGINT UNSIGNED	FK → Review NOT NULL	Review
revision_id	BIGINT UNSIGNED	FK → ReviewRevision NOT NULL	Revision
author_id	BIGINT UNSIGNED	FK → User NOT NULL	Author
parent_comment_id	BIGINT UNSIGNED	FK → ReviewComment NULLABLE	Thread parent
publish_batch_id	CHAR(36)	NULLABLE	Batch UUID
file_path	VARCHAR(1024)	NULLABLE	File (NULL = overall)
side	ENUM('left','right')	NULLABLE	Diff side
start_line	INT UNSIGNED	NULLABLE	Start line
end_line	INT UNSIGNED	NULLABLE	End line
body	TEXT	NOT NULL	Comment text
visibility	ENUM('draft','publish')	DEFAULT 'draft'	Visibility

	ed')		
is_resolved	BOOLEAN	DEFAULT FALSE	Resolved
orphaned_at	DATETIME	NULLABLE	Orphaned timestamp
created_at	DATETIME	NOT NULL	Created
published_at	DATETIME	NULLABLE	Published
resolved_at	DATETIME	NULLABLE	Resolved

INDEX: (review_id, visibility, created_at), (review_id, revision_id, file_path). Drafts visible only to author.

7.14 Notification

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
user_id	BIGINT UNSIGNED	FK → User NOT NULL	Recipient
review_id	BIGINT UNSIGNED	FK → Review NULLABLE	Related review
comment_id	BIGINT UNSIGNED	FK → ReviewComment NULLABLE	Related comment
event_type	VARCHAR(64)	NOT NULL	Event type
title	VARCHAR(512)	NOT NULL	Title
body_preview	VARCHAR(512)		Preview text
payload	JSON		Structured data
is_read	BOOLEAN	DEFAULT FALSE	Read flag
email_state	ENUM('not_applicable', 'pending', 'sent', 'failed', 'suppressed')	DEFAULT 'not_applicable'	Email status
email_attempts	TINYINT UNSIGNED	DEFAULT 0	Send attempts
sent_at	DATETIME	NULLABLE	Email sent
read_at	DATETIME	NULLABLE	Read
created_at	DATETIME	NOT NULL	Created

INDEX: (user_id, is_read, created_at), (review_id, created_at).

7.15 NotificationPreference

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
user_id	BIGINT UNSIGNED	FK → User NOT NULL	User
event_type	VARCHAR(64)	NOT NULL	Event type (* = default)
delivery_mode	ENUM('instant','digest','web_only')	NOT NULL	Mode
suppress_self	BOOLEAN	DEFAULT TRUE	Suppress own actions
created_at	DATETIME	NOT NULL	Created
updated_at	DATETIME	NOT NULL	Updated

UNIQUE: (user_id, event_type).

7.16 PathOwnerRule

Field	Type	Constraints	Description
id	BIGINT UNSIGNED	PK AUTO_INCREMENT	ID
repo_id	BIGINT UNSIGNED	FK → Repository NOT NULL	Repo
path_glob	VARCHAR(512)	NOT NULL	Glob pattern
file_ext	VARCHAR(32)	NULLABLE	Extension filter
target_user_id	BIGINT UNSIGNED	FK → User NOT NULL	Target user
action_role	ENUM('reviewer','blocking_reviewer','subscriber')	NOT NULL	Role to assign
is_active	BOOLEAN	DEFAULT TRUE	Active
created_by	BIGINT UNSIGNED	FK → User NOT NULL	Creator
created_at	DATETIME	NOT NULL	Created
updated_at	DATETIME	NOT NULL	Updated

INDEX: (repo_id, is_active). v1.2 targets individual users only. Target user must already have active UserRepository access; create otherwise fails with 422, and evaluation skips inactive or suspended users.

7.17 Entity Relationship Summary

- **User N:M Repository** via **UserRepository**
- **User 1:N Session**
- **Repository 1:N Commit**
- **Repository 1:N ContributorStat** → **ContributorIdentity 1:N ContributorAlias**
- **Repository 1:N Fork 1:N ForkChange**
- **Repository 1:N BlameCache**
- **Repository 1:N Review**
- **Review 1:N ReviewRevision**
- **Review N:M User** via **ReviewParticipant**
- **Review 1:N ReviewComment**
- **User 1:N Notification**
- **Repository 1:N PathOwnerRule**
- **Job** references Repository, Fork, User

PART IV: API & INTEGRATION

8. API Design

Internal JSON REST API. All endpoints except `/auth/*`, `/healthz`, `/readyz` require session cookie.

8.1 Conventions

Pagination: Cursor-based: `?cursor=&limit=`. Default 50, max 200. `next_cursor` in response.

Sorting: `?sort=&order=asc|desc`.

Filtering: Query params per field. Multiple values OR'd.

Async: HTTP 202 + Job body + Location header. Dedup hits use the same contract and add `X-Job-Deduplicated: true`.

Success: `{ "data": {...}, "meta": { "cursor":..., "next_cursor":..., "limit":... } }`

Error: `{ "error": { "code": "...", "message": "...", "status":... } }`

Auth failures: 401 AUTH_REQUIRED / 403 FORBIDDEN.

Jobs note: `/api/jobs` lists only user-visible operational jobs (clone, sync, fork_analysis). Notification delivery is internal and not exposed as a job resource.

8.2 Auth Endpoints

Method	Path	Response
GET	<code>/auth/github/login</code>	302 to GitHub
GET	<code>/auth/github/callback</code>	302 to app
POST	<code>/auth/logout</code>	200
GET	<code>/auth/me</code>	200 + User

8.3 Repository Endpoints

Method	Path	Response
GET	<code>/api/repos</code>	200 + paginated repos (<code>?state=active suspended all</code> ; default active)

POST	/api/repos	201 + Repository/UserRepository OR 202 + Job
GET	/api/repos/:id	200 + detail
POST	/api/repos/:id/sync	202 + Job
DELETE	/api/repos/:id	204
POST	/api/repos/:id/recheck-access	200 + updated access state

8.4 Analysis Endpoints

Method	Path	Response
GET	/api/repos/:id/commits	200 + paginated commits
GET	/api/repos/:id/commits/:sha	200 + detail
GET	/api/repos/:id/commits/:sha/diff?view=	200 + diff
GET	/api/repos/:id/contributors	200 + paginated
GET	/api/repos/:id/blame?path=&ref=	200 + blame
GET	/api/repos/:id/forks	200 + paginated forks
POST	/api/repos/:id/forks/:forkId/analyze	202 + Job
GET	/api/repos/:id/forks/:forkId/changes	200 + changes
GET	/api/repos/:id/fork-report	200 + report

8.5 Review Endpoints

Authorization notes: create review/audit = any active workspace user; add revision = author only; participant changes = author except self-subscribe; accepted/changes_requested = reviewer or blocking reviewer; close/resolve/reopen = author or blocking reviewer.

Method	Path	Description
GET	/api/repos/:id/reviews	List reviews/audits for repo
POST	/api/repos/:id/reviews	Create review/audit
GET	/api/reviews/:id	Detail + latest revision + participants
POST	/api/reviews/:id/revisions	Add new revision
GET	/api/reviews/:id/diff?revision=&view=	Diff for revision
POST	/api/reviews/:id/participants	Add participant

DELETE	/api/reviews/:id/participants/:pid	Remove participant
POST	/api/reviews/:id/comments/drafts	Create/update draft comment
GET	/api/reviews/:id/comments?visibility=	List comments
POST	/api/reviews/:id/comments/publish	Publish drafts + optional decision
POST	/api/reviews/:id/comments/:cid/resolve	Resolve comment
POST	/api/reviews/:id/comments/:cid/reopen	Reopen comment
POST	/api/reviews/:id/decision	Submit decision

8.6 Notification & Ownership Endpoints

Method	Path	Description
GET	/api/notifications	List current user notifications
POST	/api/notifications/:id/read	Mark read
POST	/api/notifications/read-all	Mark all read
GET	/api/preferences/notifications	Get prefs
PUT	/api/preferences/notifications	Upsert prefs
GET	/api/repos/:id/path-owners	List rules
POST	/api/repos/:id/path-owners	Create rule (422 if target user lacks active repo access)
DELETE	/api/repos/:id/path-owners/:ruleId	Delete rule

8.7 Job Endpoints

Method	Path	Response
GET	/api/jobs	200 + user's jobs
GET	/api/repos/:id/jobs	200 + repo jobs
GET	/api/jobs/:id	200 + detail
POST	/api/jobs/:id/cancel	200 or 403

8.8 Health Endpoints

Method	Path	Auth	Response
GET	/healthz	None	200/503 liveness
GET	/readyz	None	200/503 readiness
GET	/api/admin/diagnostics	Admin	200 + full diagnostics

9. Configuration

Viper. Precedence: env > yaml > defaults. Secrets from env only.

Key	Type	Default	Source	Description
github.client_id	string	(req)	yaml/ env	OAuth client ID
github.client_secret	string	(req)	env	OAuth secret
github.callback_url	string	(req)	yaml/ env	Redirect URI
database.host	string	localhost	yaml/ env	MySQL host
database.port	int	3306	yaml/ env	Port
database.name	string	gitlens	yaml/ env	DB name
database.user	string	(req)	yaml/ env	MySQL user
database.password	string	(req)	env	MySQL pass
server.port	int	8080	yaml/ env	HTTP port
server.session_secret	string	(req)	env	Cookie key
storage.repos_path	string	./storage/ repos	yaml/ env	Repo dir
storage.forks_path	string	./storage/	yaml/	Fork dir

h		forks	env	
encryption.key	string	(req)	env	AES-256 key
jobs.max_clone_workers	int	5	yaml/ env	Clone workers
jobs.max_analysi s_workers	int	10	yaml/ env	Analysis workers
access.recheck_i nterval	durati on	1h	yaml/ env	Access recheck
mailer.smtp_host	string	(req for email)	yaml/ env	SMTP host
mailer.smtp_port	int	587	yaml/ env	SMTP port
mailer.from_addr ess	string	(req)	yaml/ env	From address
mailer.digest_sch edule	string	0 8 * * *	yaml/ env	Digest cron
log.level	string	info	yaml/ env	Log level

PART V: CORE ALGORITHMS & SECURITY

10. Authentication Flow

1. **Login:** GET /auth/github/login → state cookie → redirect to GitHub with scope=read:user,user:email,repo
2. **Callback:** GET /auth/github/callback → validate state → exchange code for token
3. **Profile:** GET /user + GET /user/emails → primary verified email (or NULL)
4. **Upsert:** Create/update User. Encrypt token AES-256-GCM.
5. **Access sweep:** Re-validate private repositories. Accessible -> active, denied -> suspended, indeterminate -> preserve last-known state.
6. **Session:** Create in MySQL. Set signed cookie. Redirect home.
7. **Requests:** Middleware reads cookie → lookup session → inject User. Invalid = 401.
8. **Logout:** POST /auth/logout → delete session → clear cookie.

No JWTs for browser sessions. Server-side sessions for immediate revocation.

11. Fork Analysis Algorithm

11.1 Divergence Detection

1. Fetch fork HEAD and upstream HEAD SHAs.
2. Compute merge-base. Store as merge_base_sha.
3. Walk fork HEAD → merge-base = ahead_by.
4. Walk upstream HEAD → merge-base = behind_by.
5. diverged_at = author date of earliest post-merge-base fork commit.

11.2 Classification Rules

Rules in priority order. First match wins.

Pri	Rule	Type	Conf
1	Conventional: feat:, feature:	feature	0.9
2	Conventional: fix:, bugfix:, hotfix:	bugfix	0.9
3	Conventional: refactor:	refactor	0.9

4	Conventional: docs:, doc:	docs	0.9
5	Conventional: chore:, ci:, build:, deps:	chore	0.9
6	All files in docs/, README*, etc.	docs	0.8
7	All files in .github/, Makefile, etc.	chore	0.7
8	New files + ins > 3x del	feature	0.6
9	Message /fix bug patch issue #\d+/i	bugfix	0.6
10	del > ins, no new files	refactor	0.5
11	(no match)	unknown	0.3

11.3 Cherry-Pick Detection

Fork commit flagged if upstream commit has identical message body or byte-identical patch. Stored is_cherry_pick=true, excluded from unique count.

11.4 Rebase Detection

Different SHA but >95% patch overlap (Levenshtein on unified diff). Stored is_rebase=true.

11.5 Similar Changes Across Forks

Two forks: unique commits on same file path with >60% patch overlap → “potential duplicate work.”

11.6 Sync Pipeline

1. git pull --ff-only (or fetch+reset on failure).
2. Walk new HEAD to last-synced SHA. Only new commits processed.
3. Parse + INSERT commits (idempotent on repo_id, sha).
4. Incremental ContributorStat update. New ContributorIdentity/Alias as needed.
5. Update last_synced_at, clone_status='ready'.
6. Commit transaction.

Blame cache: Not mass-invalidated. SHA-keyed entries stay valid. Nightly cleanup for unreachable/stale.

Transaction: Steps 2–6 in one MySQL transaction. Git pull (step 1) is outside — pull success + DB failure is safe (next sync finds zero new commits).

12. Security

- **Token encryption:** AES-256-GCM. Key from env. Rotation = re-encrypt migration.
- **CSRF:** OAuth state param. SameSite=Lax. All mutations POST/PUT/DELETE.
- **Rate limiting:** Per-user API (100 req/min default). GitHub per-token via factory.
- **Input validation:** Repo IDs: `^[a-zA-Z0-9._-]+/[a-zA-Z0-9._-]+$`. GORM parameterized.
- **Clone isolation:** `storage/repos/{id}/`. Server-constructed paths.
- **Secrets:** All from env only.
- **HTTPS:** Behind TLS proxy (Caddy). Secure cookies.
- Private repos: Access validated on add and revalidated per FR-ACCESS-01-06. Rate-limit and transient failures are treated as indeterminate, not denial.
- **Review/audit/comment visibility:** Inherits repository access. Lost access = lost review access.
- **Private repo emails:** No raw diff hunks by default. Access rechecked before send.
- **Mentions:** Limited to users with current repository access.
- **No email reply parsing:** No email-triggered state changes in v1.2.

PART VI: OPERATIONS & DELIVERY

13. Testing Strategy

Level	Scope	Tools
Unit	Service/repo logic	testing, testify, mockgen
Integration	FX wiring, DB queries	fxtest, dockertest
API	HTTP handlers	httptest, chi helpers
Git Ops	go-git against fixtures	in-memory repos
Classification	Fork change rules	Table-driven tests
Sync Pipeline	Ordering, idempotency, rollback	dockertest, fixtures
Access Revocation	Re-validation flows	Mock GitHub API
Review Anchors	Line/hunk stability across views	Fixture diffs
Draft Publish	Visibility isolation, batch semantics	Integration tests
Revision Remap	Comment re-anchoring, orphan handling	Fixture revisions
Blocking Reviewers	State transitions, acceptance gating	Unit + integration
Notification Prefs	instant/digest/web_only, self-suppress	Unit tests
Email Security	Private-repo redaction, access recheck	Mock mailer
Path-Owner Rules	Glob matching, ext filter, dedup	Table-driven tests
E2E	OAuth → clone → analyze → review	Docker Compose
Review Authorization	Actor permissions, close/reopen, cross-user resolve	Integration tests

14. Deployment

- **Docker:** Multi-stage. Alpine + git. < 50MB.
- **Compose (dev):** App, MySQL, optional Caddy, optional MailHog for email testing.
- **Volumes:** storage/repos/, storage/forks/.
- **DB:** External MySQL in prod. Migrations on startup.

- **Proxy:** Caddy for TLS.
- **Monitoring:** /healthz, /readyz, admin diagnostics. Zap logs. Prometheus deferred.

15. Development Milestones

Phase	Scope	Key Deliverables
Phase 1: Foundation	FX scaffold, config, DB, auth, jobs, sessions	OAuth, profiles, Job entity, sessions, migrations
Phase 2: Git Core	Clone, commits, blame, diffs, sync pipeline	Repo mgmt, commit browsing, blame, sync, contributor ID
Phase 3: Fork Analysis	Fork discovery, divergence, classification	Fork list, deep analysis, classification, reports
Phase 4: Review Core	Review/audit model, endpoints, diff anchors, drafts, participants	Review CRUD, inline comments, blocking reviewers, audit-from-fork
Phase 5: Notifications & Hardening	Email, digests, subscribers, path-owners, load tests, docs	Email delivery, notification prefs, path-owner rules, production hardening

16. Open Decisions

Decision	Options	Decide By	Impact
Template engine	html/template vs Templ	Phase 1	Template DX
Diff rendering	Server HTML vs client JS	Phase 2	Frontend arch
Syntax highlighting	Chroma (Go) vs highlight.js	Phase 2	Depends on diff decision
Clone strategy for forks	Full vs shallow clone	Phase 3	Disk vs completeness
Contributor identity merging	Auto only vs manual UI	Phase 2	Accuracy across forks
Email delivery provider	SMTP vs SES/Postmark/Resend	Phase 5	Infra + deliverability
Review re-anchoring	Strict hunk match vs	Phase 4	Comment mapping accuracy

	heuristic		
Ownership routing scope	Users only vs app-local groups	Phase 5	Routing flexibility
Digest schedule	App-global daily vs user-configurable	Phase 5	UX + complexity

Appendix A: Glossary

Term	Definition
Uber FX	Go DI framework: constructor wiring, lifecycle management, modular composition
Fork	Server-side copy of a GitHub repo under different owner, linked to upstream
Merge-base	Common ancestor commit used as divergence point
Blame	Git operation: line-by-line file attribution to commits/authors
go-git	Pure Go git implementation for local ops without CLI
Conventional Commits	type: description message convention; highest classification signal
GitHubClientFactory	FX-provided factory creating per-user API clients with per-token rate limits
Review	Asynchronous code review object anchored to commits/ranges/fork deltas
Audit	A review created specifically from fork analysis results for cross-fork inspection
ReviewRevision	Immutable snapshot of a diff basis within a review
Path-owner rule	Glob-based rule auto-assigning reviewers based on changed file paths
Denormalized status	Entity field mirroring Job state for efficient queries
Cursor pagination	Opaque-token pagination stable under concurrent writes

Appendix B: v1.1 → v1.2 Changelog

1. **Code Review & Audit:** First-class Review/Audit objects with immutable ReviewRevision records, inline draft comments, threaded publish flow, resolve/reopen, participant roles (author, reviewer, blocking_reviewer, subscriber), and acceptance gating.
2. **Notifications:** In-app notifications, email delivery with per-review threading, notification preferences (instant/digest/web_only), self-suppression, subscriber follow/unfollow.

3. **Path-owner automation:** Glob-based rules auto-add participants based on changed paths and file extensions.
4. **Audit from fork analysis:** Create audits from fork-only changes, classified changes, and similarity clusters. Fork reports expose machine-readable review-suggestion objects.
5. **Private repo email security:** Access revalidated before email send. Diff hunks omitted from private-repo emails. Mentions limited to users with current access.
6. **Per-user GitHub rate limiting:** GitHubClientFactory with per-token tracking. No global startup validation.
7. **Email scope fix:** OAuth scope includes user:email. /user/emails called. Email NULLABLE.
8. **Shared job contract:** Workspace-scoped visibility, requester-only cancel, dedup always returns HTTP 202 with the existing job and is enforced by a UNIQUE dedup_key.
9. **Access revocation:** Tri-state revalidation (accessible/denied/indeterminate), no suspension on rate-limit or transient failures, suspended repositories exposed via state filter and recheck endpoint.
10. **Structural fixes:** Session locked to MySQL, frontend architecture locked to HTMX, template engine deferred separately, routes centralized in http module, auth depends on UserService.
11. **Status denormalization:** clone_status/analysis_status defined as derived from Job.status.
12. **Sync pipeline:** 6-step transactional pipeline defined. Smart blame cache (no mass-invalidation).
13. **Health split:** /healthz, /readyz (public), /api/admin/diagnostics (admin).
14. **Non-goals updated:** Added merge gating, branch protection, reply-by-email as explicit non-goals.